

Solution Requirements

Date	15 July 2026
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements

Functional Requirements (FR) define specific behaviors, functions, or features of the system (what the system should do). **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (how the system should be).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	Not applicable — the app is publicly accessible with no login or identity verification required for making a prediction.	Low
2	Authorization levels	Single anonymous user role only; no admin panel or differentiated permissions exist in the current version.	Low
3	External interfaces	No external APIs are called at runtime today. Roadmap: integrate NASA POWER / Copernicus rainfall APIs for live data ingestion.	Medium
4	Transactions processing	Each prediction is a stateless request/response transaction: form submission → validation → model inference → rendered result page. No payments or persistent transactions involved.	High
5	Reporting	The result page itself acts as an instant report: predicted risk label, confidence percentage, and a recap of all submitted rainfall values.	High
6	Business rules, etc.	Cloud Cover must be 0–100%; Annual Rainfall 0–10,000 mm; seasonal rainfall values 0–5,000/8,000 mm. Flood classification threshold is defined by the trained model, not a manual rule.	High
7	Compliance to laws or regulations	No personally identifiable information (PII) is collected or stored; only meteorological figures are processed, so no GDPR/HIPAA-scale compliance burden applies.	Medium
8	Other	Graceful error handling: invalid or missing form fields return the user to the form with a clear message instead of crashing the server.	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Model inference and page rendering must feel instant to the end user.	Prediction response returned in well under 1 second on standard hosting

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
2	Scalability	The Flask app is stateless (no session/database dependency for predictions), so it can scale horizontally.	Deployable across multiple Render instances without code changes
3	Security & Data Privacy	No PII is collected; all inputs are validated server-side to prevent malformed or malicious data from reaching the model.	100% of form submissions pass through server-side range validation before inference
4	Reliability & Availability	The app should degrade gracefully on bad input rather than crash, and remain available on its hosting platform.	Target 99% uptime on Render; friendly error page instead of a 500 crash on invalid input
5	Usability & Accessibility	Clear labeling, sufficient color contrast between text and background, and responsive layout for mobile and desktop.	Readable on common mobile and desktop viewports; clear visual distinction between flood-risk and safe states
6	Other	Maintainability: business logic (validation, prediction) is kept separate from Flask route handlers for easier testing and future extension.	Core logic can be unit-tested independently of the web layer